2026-8-4 Linux进程信号--信号产生

- 1. 信号产生的基本模型与识别
  - 1.1 信号产生的基本模型
    - 1.1.1 先有事件约定，再识别信号
    - 1.1.2 信号产生位于生命周期的起点
    - 1.1.3 所有产生路径最终都经过内核
  - 1.2 识别信号与默认动作
    - 1.2.1 信号编号与名称
    - 1.2.2 默认动作决定直接后果
    - 1.2.3 通过 signal() 观察信号是否产生
- 2. 从终端键盘产生信号
  - 2.1 Ctrl+C 产生 SIGINT
  - 2.2 Ctrl+\ 产生 SIGQUIT
  - 2.3 Ctrl+Z 产生 SIGTSTP
- 3. 从硬件异常产生信号
  - 3.1 除零产生 SIGFPE
  - 3.2 非法内存访问产生 SIGSEGV
  - 3.3 硬件异常怎样转换为进程信号
  - 3.4 Core Dump：硬件信号产生的可观察结果
- 4. 通过函数主动产生信号
  - 4.1 kill：向指定进程发送信号
  - 4.2 raise：向调用者自身发送信号
  - 4.3 abort：产生 SIGABRT 并异常终止
- 5. 从软件条件产生信号
  - 5.1 SIGPIPE：向无读端管道写入
  - 5.2 alarm：定时器到期产生 SIGALRM
  - 5.3 alarm 的一次性和剩余时间
  - 5.4 操作系统怎样管理大量定时器
- 6. 信号产生方式的全链路总结
  - 6.1 按来源归类
  - 6.2 统一产生流程
  - 6.3 判断信号来源时优先检查什么

1. 信号产生的基本模型与识别

1.1 信号产生的基本模型

1.1.1 先有事件约定，再识别信号

信号与信号量没有直接关系。信号量用于同步互斥，信号描述的是一次异步事件通知：某个事件已经发生，进程需要依据事先准备好的处理策略作出响应。
理解信号产生时，要区分三件事：

- 事件本身发生，例如按键、定时器到期、除零或非法内存访问。
- 操作系统识别事件，并确认应该通知哪个进程。
- 操作系统把相应信号写入目标进程，之后才有机会保存和处理。
  处理策略通常在信号产生之前就已确定，例如默认动作、忽略或自定义处理函数；但“已经登记处理方式”不代表信号已经产生，也不代表处理函数已经被调用。

1.1.2 信号产生位于生命周期的起点

一个信号通常经历：

```
信号产生 -> 信号保存 -> 信号处理
```

本笔记的终点是“信号产生并经内核记录”。信号不会总能在事件发生的瞬间被处理，因此内核需要先保存它，再选择合适时机递送。

1.1.3 所有产生路径最终都经过内核

键盘、命令、函数、软件条件和硬件异常看起来来源不同，但最终都不是普通用户进程直接修改另一个进程的数据：

```
外部事件 / 其他进程 / 命令 / 系统调用 / 软件条件 / 硬件异常
                          |
                          v
                    操作系统内核
                          |
                          v
              给目标进程写信号待处理信息
```

信号待处理信息位于进程的内核管理数据中，可以抽象理解为 `task_struct` 中的位图：

- 位的位置对应信号编号。
- 位的值表示该信号是否已经到达待处理状态。

- 只有内核有资格直接修改这些数据。
  因此，“向进程发送信号”的本质，是借助系统调用或内核内部路径，让操作系统向目标进程写入信号，而不是调用者直接访问目标进程的内核结构。

1.2 识别信号与默认动作

1.2.1 信号编号与名称

每个信号都有编号和宏名称。程序可以使用编号，也可以使用名称，例如：

```
2       SIGINT
3       SIGQUIT
6       SIGABRT
8       SIGFPE
11      SIGSEGV
14      SIGALRM
```

常见 Linux 环境中的编号划分如下：

- 1-31：标准信号。
- 34-64：实时信号。
- 32、33：通常保留或被系统使用。
  普通信号列表中没有 0 号信号。`kill(pid, 0)` 可以用于检查进程是否存在并检查发送权限，但它不会向目标进程投递一个编号为 0 的信号。
  查看当前系统的信号编号和名称：

```
kill -l
```

```
ZL@VM-0-14-ubuntu:~/code$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

具体编号可能随体系结构变化，应以本机 `kill -l` 和 `man 7 signal` 为准。

1.2.2 默认动作决定直接后果

同一个“终止”大类中，`Term` 和 `Core` 仍有所不同：

- `Term`：终止进程。
- `Core`：终止进程，并允许生成 core dump。
- `Stop`：暂停进程。
- `Cont`：继续运行。
- `Ign`：忽略。
  与信号产生相关的重点信号如下：

编号	名称	默认动作	典型产生条件
2	`SIGINT`	Term	终端按下 `Ctrl+C`
3	`SIGQUIT`	Core	终端按下 `Ctrl+\`
6	`SIGABRT`	Core	调用 `abort()`
8	`SIGFPE`	Core	CPU 检测到算术异常
9	`SIGKILL`	Term	`kill` 发送，不能被捕获或忽略
11	`SIGSEGV`	Core	MMU/页表发现非法内存访问
13	`SIGPIPE`	Term	向没有读端的管道或套接字写入
14	`SIGALRM`	Term	`alarm()` 定时器到期
15	`SIGTERM`	Term	`kill` 或程序请求正常终止
18	`SIGCONT`	Cont	继续运行被暂停的进程
19	`SIGSTOP`	Stop	强制暂停，不能被捕获或忽略
20	`SIGTSTP`	Stop	终端按下 `Ctrl+Z`

查看默认动作和信号含义：

```
man 7 signal
```

Signal	Standard	Action	Comment
SIGABRT	P1990	Core	Abort signal from **abort**(3)
SIGALRM	P1990	Term	Timer signal from **alarm**(2)
SIGBUS	P2001	Core	Bus error (bad memory access)
SIGCHLD	P1990	Ign	Child stopped or terminated
SIGCLD	-	Ign	A synonym for **SIGCHLD**
SIGCONT	P1990	Cont	Continue if stopped
SIGEMT	-	Term	Emulator trap
SIGFPE	P1990	Core	Floating-point exception
SIGHUP	P1990	Term	Hangup detected on controlling terminal or death of controlling process
SIGILL	P1990	Core	Illegal Instruction
SIGINFO	-		A synonym for **SIGPWR**
SIGINT	P1990	Term	Interrupt from keyboard
SIGIO	-	Term	I/O now possible (4.2BSD)
SIGIOT	-	Core	IOT trap. A synonym for **SIGABRT**
SIGKILL	P1990	Term	Kill signal
SIGLOST	-	Term	File lock lost (unused)
SIGPIPE	P1990	Term	Broken pipe: write to pipe with no readers; see **pipe**(7)
SIGPOLL	P2001	Term	Pollable event (Sys V); synonym for **SIGIO**
SIGPROF	P2001	Term	Profiling timer expired
SIGPWR	-	Term	Power failure (System V)
SIGQUIT	P1990	Core	Quit from keyboard
SIGSEGV	P1990	Core	Invalid memory reference
SIGSTKFLT	-	Term	Stack fault on coprocessor (unused)
SIGSTOP	P1990	Stop	Stop process
SIGTSTP	P1990	Stop	Stop typed at terminal
SIGSYS	P2001	Core	Bad system call (SVr4);

1.2.3 通过 signal() 观察信号是否产生

为了验证某个动作是否确实产生了指定信号，可以先用 `signal()` 登记一个处理函数，再触发对应事件。`signal()` 本身只登记处理方式，不会主动产生信号。

```
#include <iostream>
#include <signal.h>
#include <unistd.h>

void handler(int signo)
{
    std::cout << "catch signal: " << signo << std::endl;
}

int main()
{
    signal(SIGINT, handler);

    while (true)
```

```
    {
        std::cout << "process: " << getpid() << std::endl;
        sleep(1);
    }
}
```

上面的例子登记了 `SIGINT` 的处理函数。运行后按 `Ctrl+C`，如果打印出信号编号 2，就说明终端按键已经转换为 `SIGINT` 并送达进程。

```
signal(2)

NAME
       signal - ANSI C signal handling

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <signal.h>

       typedef void (*sighandler_t)(int);

       sighandler_t signal(int signum, sighandler_t handler);
```

```
drwxrwxr-x  2 ZL ZL  4096 Aug  4 23:44 ./
drwxrwxr-x 30 ZL ZL  4096 Aug  4 21:43 ../
-rw-rw-r--  1 ZL ZL    54 Aug  4 22:01 makefile
-rwxrwxr-x  1 ZL ZL 16720 Aug  4 23:44 testsig*
-rw-rw-r--  1 ZL ZL   438 Aug  4 23:44 testsig.cc
ZL@VM-0-14-ubuntu:~/code/lesson26$ kill -2 2237371
ZL@VM-0-14-ubuntu:~/code/lesson26$
```

```
test sig...,pig 2237371
test sig...,pig 2237371
test sig...,pig 2237371
catch signal: 2
test sig...,pig 2237371
test sig...,pig 2237371
test sig...,pig 2237371
```

2. 从终端键盘产生信号

2.1 Ctrl+C 产生 SIGINT

在终端运行前台进程时按下 `Ctrl+C`，内核会把这种终端输入解释为 `SIGINT`（2）。如果进程没有改变处理方式，默认动作是终止进程。
所以 Shell 最终显示“程序被终止”，并不是键盘字符被程序直接读取后自行退出，而是“键盘事件 -> 内核识别组合键 -> 向目标进程写 `SIGINT` -> 进程执行默认动作”的结果。

2.2 Ctrl+\ 产生 SIGQUIT

按下 `Ctrl+\` 会产生 `SIGQUIT`（3）。其默认动作是 `Core`：先终止进程，并允许系统将异常现场保存为 core 文件。
`SIGQUIT` 与 `SIGINT` 都能终止前台进程，区别在于前者默认带有 core dump 语义。

2.3 Ctrl+Z 产生 SIGTSTP

按下 `Ctrl+Z` 会产生 `SIGTSTP`（20），默认动作是暂时停止进程。常见 Linux 环境中：

```
Ctrl+Z -> SIGTSTP(20) -> Stop
kill -18 PID -> SIGCONT(18) -> Cont
```

被暂停的进程没有退出。它可以在收到 `SIGCONT` 后继续运行，并由 Shell 的作业控制决定放到前台还是后台。
终端组合键产生的信号面向当前前台进程组。使用 `&` 放到后台的进程不能像前台进程一样直接接收这些键盘控制组合键；查看作业状态可以用：

```
jobs
```

要终止后台进程，需要按 PID 或作业编号向它发送信号。

3. 从硬件异常产生信号

3.1 除零产生 SIGFPE

程序执行整数除零时，CPU 会在硬件层发现算术异常。内核识别当前进程执行了故障指令，并把该异常转换为 `SIGFPE`（8）发送给当前进程。

```
#include <signal.h>
#include <unistd.h>

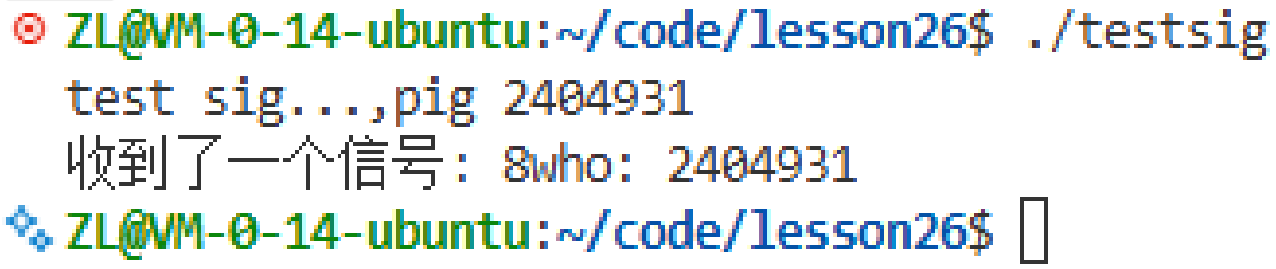
void handler(int signo)
{
    (void)signo;
}

int main()
{
    signal(SIGFPE, handler);

    while (true)
    {
        sleep(1);
        int a = 10;
        a /= 0;
```

```
    }
}
```

`SIGFPE` 的名字虽然是 floating-point exception，但 Linux 也用它表示整数除零等算术异常。



3.2 非法内存访问产生 SIGSEGV

程序解引用空指针或访问未映射地址时，MMU 和页表无法完成地址转换或权限检查，于是产生内存访问异常。内核将其转换为 `SIGSEGV`（11）发送给触发异常的进程。

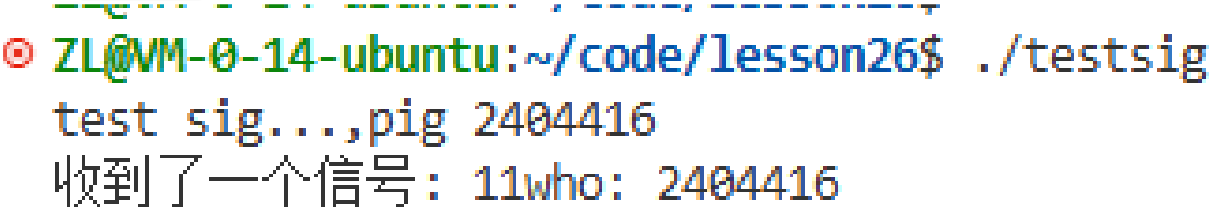
```
#include <signal.h>
#include <unistd.h>

void handler(int signo)
{
    (void)signo;
}

int main()
{
    signal(SIGSEGV, handler);

    sleep(1);
    int *p = nullptr;
    *p = 100;
}
```

捕获 `SIGSEGV` 只改变了进程对异常信号的响应方式，并没有自动修复无效地址。



3.3 硬件异常怎样转换为进程信号

除零和非法内存访问是同步异常：它们由当前执行的指令直接触发，因此操作系统能够把异常归属于“当前正在运行的进程”。
可以概括为下面的路径：

```
当前进程执行指令
        |
        +-- CPU 检测算术异常 ----------> SIGFPE(8)
        |
        +-- MMU 检测地址/权限异常 -----> SIGSEGV(11)
        |
        v
内核定位当前进程
        |
        v
向当前进程写入待处理信号
```

CPU 控制状态、寄存器和异常标志属于当前进程的硬件上下文。若自定义处理函数只是打印信息后返回，却没有修复故障条件，CPU 可能再次执行同一条故障指令并再次产生同一信号，表现为持续输出或死循环。

3.4 Core Dump：硬件信号产生的可观察结果

`SIGFPE`、`SIGSEGV`、`SIGQUIT`、`SIGABRT` 等信号的默认动作是 `Core`。它们与 `Term` 一样都会终止进程，但 `Core` 还允许系统把进程异常退出时的内存现场写入 core 文件，用于事后分析。

```
Term：终止进程
Core：终止进程 + 允许核心转储
```

```
ZL@VM-0-14-ubuntu:~/code/lesson26$ ./testsig
test sig...,pig 2397460
Segmentation fault (core dumped)
ZL@VM-0-14-ubuntu:~/code/lesson26$
```

core 文件是否真正生成，还受资源限制影响。查看限制：

```
ulimit -a
```

允许生成 core 文件：

```
ulimit -c 10240
```

关闭核心转储：

```
ulimit -c 0
```

```
ZL@VM-0-14-ubuntu:~$ ulimit -a
real-time non-blocking time  (microseconds, -R) unlimited
core file size              (blocks, -c) 0
data seg size               (kbytes, -d) unlimited
scheduling priority                 (-e) 0
file size                   (blocks, -f) unlimited
pending signals                     (-i) 7544
max locked memory           (kbytes, -l) 251856
max memory size             (kbytes, -m) unlimited
open files                          (-n) 1024
pipe size                (512 bytes, -p) 8
POSIX message queues         (bytes, -q) 819200
real-time priority                  (-r) 0
stack size                  (kbytes, -s) 8192
cpu time                   (seconds, -t) unlimited
max user processes                  (-u) 7544
virtual memory              (kbytes, -v) unlimited
file locks                          (-x) unlimited
ZL@VM-0-14-ubuntu:~$
```

当终端显示 `Floating point exception (core dumped)` 时，表示子进程因信号异常终止，并且父进程收到的退出状态中带有 core dump 标志。Shell 并不处理程序的业务错误，它只是通过子进程退出状态告诉用户“由哪个信号终止、是否发生了核心转储”。
调试 core 文件时，可在调试器中加载现场：

```
core-file core.<编号>
```

服务器常默认关闭 core dump，主要是为了避免异常重启时持续堆积大文件；core 文件也可能包含敏感内存内容。

4. 通过函数主动产生信号

4.1 kill：向指定进程发送信号

Shell 命令：

```
kill -[信号编号或名称] [进程PID]
```

例如：

```
kill -15 12345
kill -SIGTERM 12345
```

`kill` 命令最终通过内核的发送接口完成投递。对应的 C 接口是：

```
#include <sys/types.h>
#include <signal.h>

int kill(pid_t pid, int sig);
```

可以向另一个进程发送指定编号的信号。成功发送时返回 0，失败时返回 -1 并设置 `errno`。

```
NAME
       kill - send signal to a process

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <signal.h>

       int kill(pid_t pid, int sig);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       kill():
           _POSIX_C_SOURCE
```

可以用 `kill()` 实现一个简化版 `mykill`：

```
#include <iostream>
#include <string>
#include <signal.h>

void Usage(const char *cmd)
{
    std::cout << "Usage: " << cmd << " signumber who" << std::endl;
}
```

```
int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        Usage(argv[0]);
        return 1;
    }

    int signumber = std::stoi(argv[1]);
    pid_t pid = std::stoi(argv[2]);
    return kill(pid, signumber) == 0 ? 0 : 1;
}
```

被发送信号的动作仍由目标进程决定：同一个 `SIGTERM` 可能终止默认进程，也可能被目标进程捕获或忽略。

4.2 raise：向调用者自身发送信号

`raise()` 用于给调用它的进程或线程发送信号，概念上相当于：

```
raise(sig);
kill(getpid(), sig);
```

```
#include <iostream>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    std::cout << "caught: " << sig << " pid: " << getpid() << std::endl;
}

int main()
{
    signal(SIGINT, handler);

    while (true)
    {
        sleep(2);
        raise(SIGINT);
    }
}
```

`raise()` 成功时返回 0，失败时返回非零值。

```
raise(3)

NAME
       raise - send a signal to the caller

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <signal.h>

       int raise(int sig);
```

4.3 abort：产生 SIGABRT 并异常终止

`abort()` 用于让当前进程异常终止，通常会向调用者发送 `SIGABRT` (6)：

```c
#include <stdlib.h>

void abort(void);
```

概念上可以把核心动作理解为：

```c
kill(getpid(), SIGABRT);
```

但 `abort()` 的完整行为比单独调用一次 `kill()` 更强：它不会正常返回。即使 `SIGABRT` 被自定义处理函数捕获，处理函数返回后，`abort()` 仍会继续异常终止流程，因此终端通常同时看到用户处理函数的输出和 `Aborted`。

```
NAME
       abort - cause abnormal process termination

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <stdlib.h>

       [[noreturn]] void abort(void);
```

下面的示例展示了“捕获到了 6 号信号，但进程仍然被终止”：

```cpp
#include <iostream>
#include <signal.h>

void handler(int sig)
{
    std::cout << "caught: " << sig << std::endl;
}

int main()
{
    signal(SIGABRT, handler);
    abort();
}
```

```
ZL@VM-0-14-ubuntu:~/code/lesson27$ ./mykill
进程正在运行：  2727940
进程捕捉到信号：  6 pid: 2727940
Aborted
```

5. 从软件条件产生信号

5.1 SIGPIPE：向无读端管道写入

当进程继续向一个已经没有读端的管道或套接字写入数据时，内核能够识别这个软件条件，并向写入进程发送 `SIGPIPE`（13）。其默认动作是终止进程。
这类信号不是由 CPU 算术异常或 MMU 内存异常触发，而是由内核检测到的软件和系统对象状态触发。

5.2 alarm：定时器到期产生 SIGALRM

```
#include <unistd.h>

unsigned int alarm(unsigned int seconds);
```

调用 `alarm(seconds)` 后，内核会在约 `seconds` 秒后给调用进程发送 `SIGALRM`（14）。默认动作是终止进程。

```
NAME
       alarm - set an alarm clock for delivery of a signal

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       unsigned int alarm(unsigned int seconds);
```

验证 `SIGALRM` 是否产生：

```
#include <iostream>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    std::cout << "catch signal: " << sig << std::endl;
}

int main()
{
    signal(SIGALRM, handler);
    alarm(1);
    sleep(2);
}
```

```
进程捕捉到信号： 14 pid: 2740443
```

用 `alarm(1)` 观察一秒内能执行多少次循环：

```
int count = 0;
alarm(1);

while (true)
{
    std::cout << "count: " << count << std::endl;
    ++count;
}
```

加入屏幕输出以后，一秒内通常只能完成较少次计数；去掉输出后，计数次数会明显增加。这并不是信号产生了“不同数量的时间”，而是终端 I/O 很慢，消耗了大量 CPU 执行时间。该实验同时说明了 I/O 和内核缓冲区对效率的影响。

```
ZL@VM-0-14-ubuntu:~/code/lesson27$ make
g++ -w -o mykill mykill.cc
ZL@VM-0-14-ubuntu:~/code/lesson27$ ./mykill
进程捕捉到信号：  14 pid: 2741195 cnt: 560107329
ZL@VM-0-14-ubuntu:~/code/lesson27$ ./mykill
进程捕捉到信号：  14 pid: 2741399 cnt: 561749114
```

5.3 alarm 的一次性和剩余时间

`alarm()` 设置的是一次性定时器。要在每次超时后继续触发，应在处理函数中重新设置：

```
#include <signal.h>
#include <unistd.h>

void on_alarm(int sig)
{
    (void)sig;
    alarm(1);
}

int main()
{
    signal(SIGALRM, on_alarm);
    alarm(1);

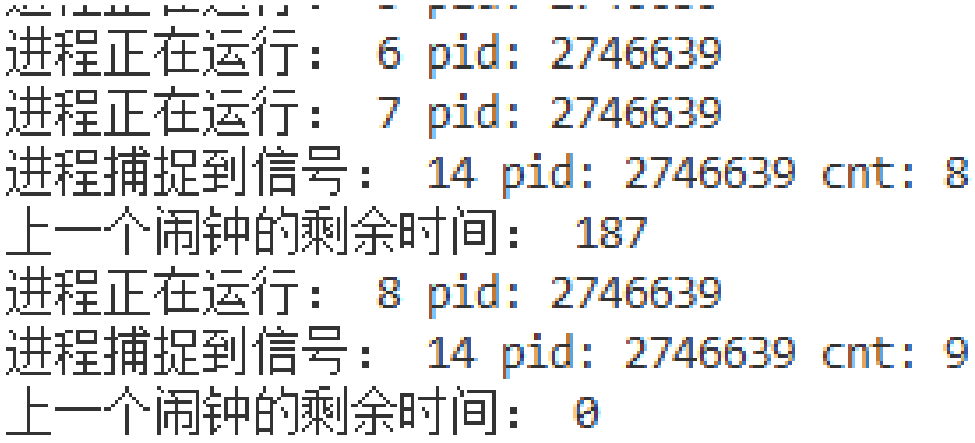
    while (true)
        pause();
}
```

`alarm()` 的返回值是“之前已经设置的闹钟还剩多少秒”；若此前没有闹钟，则返回 0：

```
alarm(5) -> 0
等待 2 秒后
alarm(10) -> 3
```

新调用会替换旧闹钟。如果把 `alarm()` 放进一个执行得非常快的 `while` 循环中，每轮都可能取消并重新设置定时器，原来的闹钟便一直来不及到期。
`alarm(0)` 表示取消当前闹钟：

```
int remaining = alarm(0);
```



5.4 操作系统怎样管理大量定时器

每个闹钟本质上都是一项定时任务。操作系统管理它们的基本思路是“先描述，再组织”：

- 描述到期时间。
- 描述是否重复以及触发次数。
- 描述到期后执行什么动作。
- 按到期先后组织所有任务，并处理最早到期的任务。
  可以用最小堆帮助理解“优先检查最早到期项”的思路，但课程中的这种模型不等于 Linux 内核的完整定时器实现；本文只保留“定时器到期 -> 产生 `SIGALRM` ”这条产生链路。

6. 信号产生方式的全链路总结

6.1 按来源归类

来源类别	触发者	典型例子	产生的信号
终端键盘	用户或终端驱动	`Ctrl+C`、`Ctrl+\`、`Ctrl+Z`	`SIGINT`、`SIGQUIT`、`SIGTSTP`
系统命令	用户	`kill -15 PID`	指定的信号
系统调用	程序开发者在代码中调用	`kill`、`raise`、`abort`	指定信号或 `SIGABRT`
硬件异常	CPU、MMU 等硬件配合内核	除零、非法内存访问	`SIGFPE`、`SIGSEGV`
软件条件	内核检测软件/系统状态	无读端管道、定时器到期	`SIGPIPE`、`SIGALRM`

6.2 统一产生流程

```
事件发生
    |
    v
终端驱动 / 其他进程 / 用户命令 / 系统调用 / 硬件异常 / 软件条件
    |
    v
操作系统确认事件与目标进程
    |
    v
内核向目标进程写入待处理信号
    |
    +--> 信号产生阶段结束
         |
         v
       信号保存与处理（后续主题）
```

无论入口多么不同，最终都汇聚到“内核给目标进程写信号”。普通用户程序不能绕过内核直接修改目标进程的信号待处理位图。

6.3 判断信号来源时优先检查什么

- 键盘组合键只作用于前台进程组；后台进程需要按 PID 或作业编号发送信号。
- `Ctrl+C`、`Ctrl+\`、`Ctrl+Z` 分别对应 `SIGINT`、`SIGQUIT`、`SIGTSTP`，常见编号为 2、3、20。

- `SIGQUIT`、`SIGFPE`、`SIGSEGV`、`SIGABRT` 默认属于 `Core`，可能产生 core 文件。
- core 文件是否落地还取决于 `ulimit -c`、权限和系统配置。
- `kill()` 可以指定目标进程，`raise()` 面向调用者自身，`abort()` 固定产生 `SIGABRT` 并最终异常终止。
- `SIGPIPE` 和 `SIGALRM` 由内核检测软件条件产生，不是由 CPU 直接报错。
- 硬件异常被转换为信号以后，如果处理函数没有修复原始故障，同一故障可能再次产生同一信号。